

# EXPERIMENT 01:

1. Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

## PSEUDO CODE :

Start

Call fork() to create a child process

If fork() returns 0

    This is Child Process

    Get PID using getpid()

    Get PPID using getppid()

    Display Child PID and PPID

Else if fork() > 0

    This is Parent Process

    Get PID using getpid()

    Get PPID using getppid()

    Display Parent PID and PPID

Else

    Display "Fork Failed"

Stop

SAMPLE OUTPUT :

Parent Process

PID : 4521

PPID : 3312

Child Process

PID : 4522

PPID : 4521

## CODE:

```
#include <stdio.h>

#include <unistd.h>


int main()
{
    pid_t pid;


    pid = fork();


    if(pid == 0)
    {
        printf("\nChild Process");

        printf("\nPID : %d", getpid());

        printf("\nPPID : %d\n", getppid());
    }
    else if(pid > 0)
    {
        printf("\nParent Process");

        printf("\nPID : %d", getpid());

        printf("\nPPID : %d\n", getppid());
    }
    else
    {
        printf("Fork Failed");
    }


    return 0;
}
```

codechef.com/c-online-compiler

C

Run

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main()
5 {
6     pid_t pid;
7
8     pid = fork();
9
10    if(pid == 0)
11    {
12        printf("\nChild Process");
13        printf("\nPID : %d", getpid());
14        printf("\nPPID : %d\n", getppid());
15    }
16    else if(pid > 0)
17    {
18        printf("\nParent Process");
19        printf("\nPID : %d", getpid());
20        printf("\nPPID : %d\n", getppid());
21    }
22    else
23    {
24        printf("Fork Failed");
25    }
26
27    return 0;
28 }
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:0.0000 secsMemory:1.768 Mb

Your Output

Parent Process

PID : 1601

PPID : 1600

Child Process

PID : 1602

PPID : 1